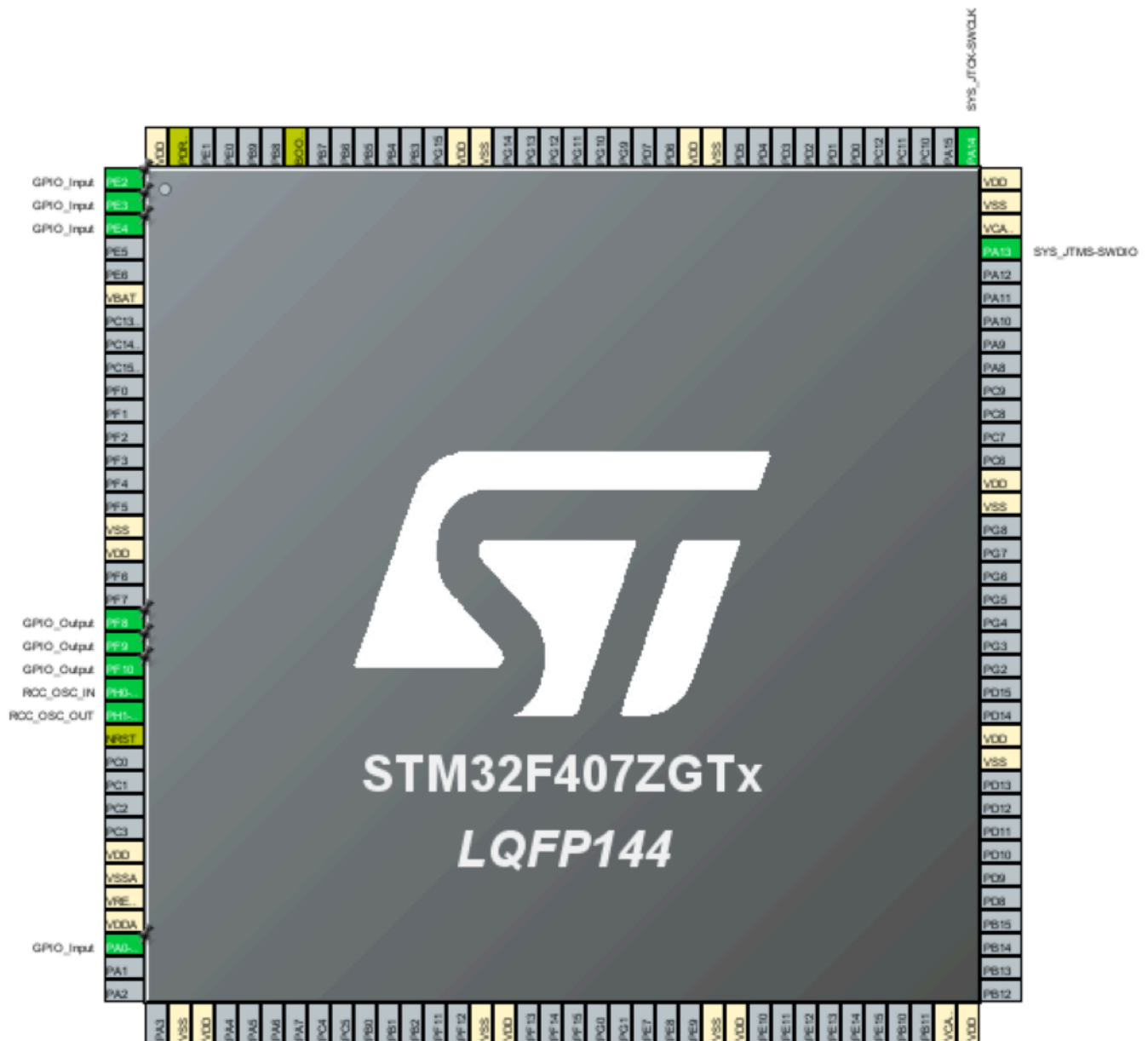


项目说明： 按键控制LED和蜂鸣器

实现了通过四个按键（左、下、右、上）控制LED灯和蜂鸣器。

主要功能实现



- ### 1. 左按键 (GPIOE, GPIO_PIN_4):

- 当左按键按下时（检测到 GPIOE, GPIO_PIN_4 为低电平，并经过20ms消抖），控制 GPIOF, GPIO_PIN_9 上的LED亮起（GPIOF, GPIO_PIN_9 输出低电平）。
- 松开按键时，LED熄灭（GPIOF, GPIO_PIN_9 输出高电平）。

2. 下按键 (GPIOE, GPIO_PIN_3):

- 当下按键按下时（检测到 GPIOE, GPIO_PIN_3 为低电平，并经过20ms消抖），控制 GPIOF, GPIO_PIN_8 上的蜂鸣器发声（GPIOF, GPIO_PIN_8 输出低电平）。
- 松开按键时，蜂鸣器停止（GPIOF, GPIO_PIN_8 输出高电平）。

3. 右按键 (GPIOE, GPIO_PIN_2):

- 当右按键按下时（检测到 GPIOE, GPIO_PIN_2 为低电平，并经过20ms消抖），控制 GPIOF, GPIO_PIN_10 上的LED亮起（GPIOF, GPIO_PIN_10 输出低电平）。
- 松开按键时，LED熄灭（GPIOF, GPIO_PIN_10 输出高电平）。

4. 上按键 (GPIOA, GPIO_PIN_0):

- 当上按键按下时（检测到 GPIOA, GPIO_PIN_0 为高电平，并经过20ms延时），同时控制 GPIOF, GPIO_PIN_8 (蜂鸣器), GPIOF, GPIO_PIN_9 (LED1), GPIOF, GPIO_PIN_10 (LED2) 亮起/发声（全部输出低电平）。

实现方式

• 主程序 (main.c):

- 初始化HAL库和系统时钟。
- 调用 MX_GPIO_Init() 配置GPIO引脚。
-
- 进入一个无限循环 while(1)，在该循环中持续调用 HardWare.c 中定义四个按键处理函数：left_key_process()、down_key_process()、right_key_process() 和 up_key_process()。

• 分文件编写 (HardWare.c, HardWare.h):

- 包含了四个按键处理函数，每个函数负责检测一个特定按键的状态。
- 通过 HAL_GPIO_ReadPin() 读取按键连接的GPIO引脚状态。
- 为了消除按键抖动，在检测到按键按下后，会引入一个 HAL_Delay(20) 的延时，并再次确认按键状态。
- 通过 HAL_GPIO_WritePin() 控制与LED和蜂鸣器连接的GPIO引脚的输出电平，从而实现对它们的开关控制。

代码已上传至 [github](#)